

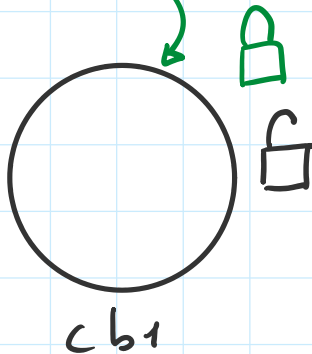
Un metodo synchronized cerca di prendere il lock dell'oggetto su cui viene invocato

Supponiamo di avere

```
ContoBancario2 cb1 = new ContoBancario2( -- );
ContoBancario2 cb2 = new ContoBancario2( -- );
```

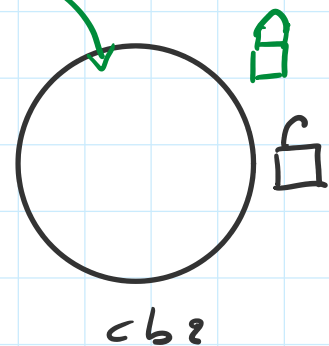
t1:

cb1.preleva();



t2:

cb2.preleva();



Oggetti diversi → lock diversi

I lock java sono rientranti :

se un thread possiede il lock di un oggetto e prova a riprenderlo non si blocca

oggetto e prova a riprenderlo non si blocca

Esempio:

```
public class A {
```

```
    public synchronized void m1() {
```

```
        ≡
```

```
        m2();
```

```
        ≡
```

```
    }
```

```
    private synchronized void m2() {
```

```
        ≡
```

```
    }
```

```
}
```

il lock viene rilasciato al termine di m1()

l'acquisizione del lock avviene senza problemi (non c'è blocco)

t1:

```
A a = new A();
```

```
a.m1();
```

Nel caso di metodi `synchronized` il lock viene rilasciato sia

- in caso di esecuzione normale, quando termina l'esecuzione del corpo del metodo
- in caso di eventi anomali (eccezioni), quando si esce dal metodo.

Blocchi synchronized

la forma è

```
synchronized (oggetto) {
```

```
==
```

```
}
```

oggetto di cui vogliamo prendere il lock

blocco di istruzioni protetto dal lock in questione

Quando scriviamo

```
class B {
```

```
    synchronized void m1() {
```

```
        ==
```

```
    }
```

```
    synchronized void m2() {
```

```
        ==
```

```
    }
```

```
}
```

Forma da preferire:
più leggibile e sintetica

è lo stesso che scrivere

```
class B {
```

```
    void m1() {
```

oggetto implicato

```

void m1() {
    synchronized (this) {
        // ...
    }
}

void m2() {
    synchronized (this) {
        // ...
    }
}

```

oggetto implemento

↙

Ci sono alcune situazioni in cui non possiamo fare a meno dei blocchi synchronized

- 1) non tutto il codice di un metodo deve essere eseguito in mutua esclusione
- 2) sincronizzazione su oggetti diversi dell'oggetto implemento
- 3) sincronizzazione su un array

1) public void m() {

```

    {
        synchronized (this) {
            // ...
        }
    }

```

↗

solo questo
in
mutua esclusione

$\left. \begin{array}{l} \{ \\ \equiv \\ \} \end{array} \right\} \text{in} \\ \text{mutua esclusione}$
 $\left. \begin{array}{l} \{ \\ \equiv \\ \} \end{array} \right\} \text{Possono essere eseguite "in} \\ \text{contemporanea" con altri} \\ \text{thread}$

2)

$\text{void } m(\text{Parametro } p) \{$
 $\equiv$
 $\text{synchronized}(p) \{$
 $\text{Codice che modifica } p$
 $\}$
 $\equiv$
 $\}$
 $\}$

l'oggetto
 di cui si
 prende il
 lock è p

3)

$\text{int}[] \ v = \text{new int}[-];$

$t_1:$

$\text{synchronized}(v) \{$
 $\equiv$
 $\}$

$t_2:$

$\text{synchronized}(v) \{$
 $\equiv$
 $\}$

}

}

Esempio, più lock, diversi dall'oggetto impilato:

```
class Separati {
```

```
    private int v1;
```

```
    private int v2;
```

```
    protected final Object l1 = new Object();
```

```
    protected final Object l2 = new Object();
```

```
    public int getV1() {
```

```
        synchronized (l1) {
```

```
            return v1;
```

```
        }
```

```
    }
```

← Prendo solo il lock l1

```
    public void setV1(int v) {
```

```
        synchronized (l1) {
```

```
            v1 = v;
```

```
        }
```

```
    }
```

```
    public int getV2() {
```

```
        synchronized (l2) {
```

```
            return v2;
```

```
        }
```

```
    }
```

```
    public void setV2(int v) {
```

```
        synchronized (l2) {
```

```
            v2 = v;
```

```
        }
```

```
    }
```

← Qui prendo solo il lock l2

```
    public void makeEqual() {
```

```
        synchronized (l1) {
```

← Qui prendo solo il lock l1

```

    synchronized (l1) {
        synchronized (l2) {
            v1 = v2;
        }
    }
}

```

Qui prendo
entrambi i
lock

Posso eseguire "in contemporanea"

- get v1() con set v2() e get v2()
- set v1() " " "
- get v2() " set v1() e get v1()
- ;

Questo aumenta il livello di parallelismo del codice

C'è mutua esclusione tra

- get v1 e set v1 e makeEqual
- set v2 e get v2 e makeEqual
- ;

Metodi statici sincronizzati

Quando un metodo statico è indicato come synchronized il lock è quello della classe (e non delle istanze della classe)

Questo significa che due thread non possono eseguire "in contemporanea" metodi statici sincronizzati sulla stessa classe.

Esempio:

```
public class ContoBancario {  
    private int numero;  
    private static int prossimoNumero;  
  
    public ContoBancario() {  
        numero = prossimoNumero++;  
    }  
    ;  
}
```

Se più thread possono creare istanze di Conto Bancario possono venire fuori inconsistenze

Soluzione: accediamo al campo statico prossimoNumero con un metodo statico synchronized.

```
public class ContoBancario {  
    private int numero;  
    private static int prossimoNumero;
```

```

public ContoBancario () {
    numero = getNextNumero();
}
private static synchronized int
    getNextNumero () {
        return prossimoNumero++;
    }
;
}

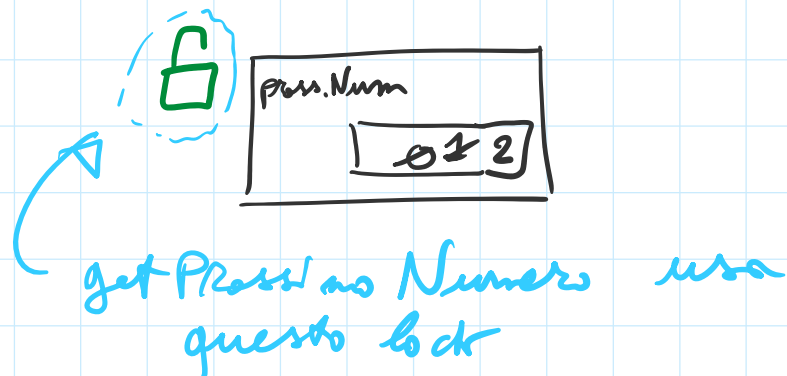
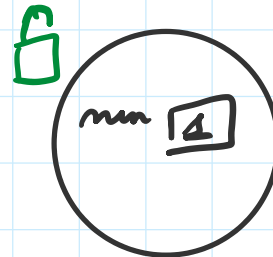
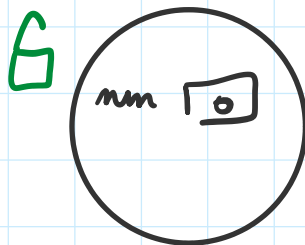
```

t1:

t2:

ContoBancario c1 = new ContoBancario();

ContoBancario c2 = new
ContoBancario();



Ci sono tre lock!

Deadlock

Una situazione di blocco da cui non si riesce più ad uscire

Per potersi verificare ci devono essere almeno due thread e due lock

Per esempio

t₁
⋮
acquista l₁
⋮
tenta di
acquistare l₂
↓
si blocca

t₂
⋮
acquista l₂
⋮
tenta di acquistare l₁
↓
si blocca

```
.....
Lancia.java
.....
public class Lancia {
    public static void main(String[] args){
        StatoGioco s = new StatoGioco();
        ViteRimaste v = new ViteRimaste();
        Mario m = new Mario(s, v);
        Luigi l = new Luigi(s, v);
        m.start();
        l.start();
    }
}
.....
Luigi.java
.....
public class Luigi extends Thread {

    private StatoGioco sg;
    private ViteRimaste vr;

    public Luigi(StatoGioco sg, ViteRimaste vr) {
        this.sg = sg;
        this.vr = vr;
    }
    public void run(){
        while(true) {
            // altro codice
            vr.controllaQuante(sg);
        }
    }
}
```

```

}
.....
Mario.java
.....
public class Mario extends Thread {

    private StatoGioco sg;
    private ViteRimaste vr;

    public Mario(StatoGioco sg, ViteRimaste vr) {
        this.sg = sg;
        this.vr = vr;
    }
    public void run(){
        while(true) {
            // altro codice
            sg.controllaPunteggio(vr);
        }
    }
}
.....
StatoGioco.java
.....
public class StatoGioco {

    private boolean fine;

    public synchronized void controllaPunteggio(ViteRimaste vr){
        System.out.println(Thread.currentThread().getName() + " chiamato getPunteggio()");
        // il seguente metodo potrebbe essere chiamato quanto il punteggio
        // supera una certa soglia. Qui lo chiamiamo sempre per sollecitare
        // il problema
        vr.incrementa();
    }

    public synchronized void termina(){
        fine = true;
        System.out.println(Thread.currentThread().getName() + " chiamato termina()");
    }
}
.....
ViteRimaste.java
.....
public class ViteRimaste {

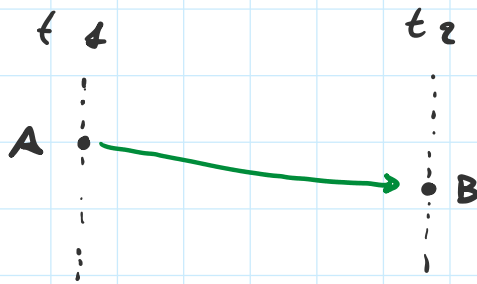
    private int q;

    public synchronized void incrementa(){
        // Anche qui supponiamo che ci sia del codice opportuno
        System.out.println(Thread.currentThread().getName() + " chiamato incrementa()");
    }

    public synchronized void controllaQuante(StatoGioco sg) {
        System.out.println(Thread.currentThread().getName() + " chiamato quante()");
        // termina() potrebbe essere chiamato quando il numero di vite diventa uguale a zero
        sg.termina();
    }
}

```

Problema :



Voglio eseguire l'operazione B in t_2 solo dopo che t_1 ha eseguito A

Comunicazione tra thread

- nel mondo della programmazione concorrente si detta sincronizzazione

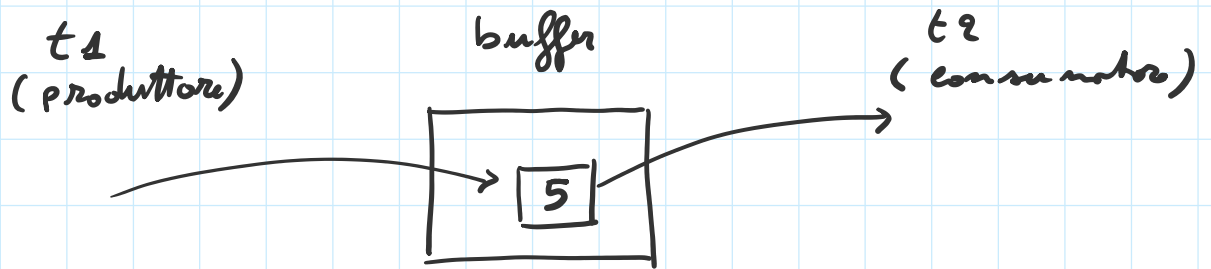
Il costrutto `synchronized` risolve i problemi di interferenza (mediante esecuzione in mutua esclusione).

I problemi di comunicazione (sincronizzazione) vengono risolti con i metodi

- `wait()`
- `notify()`
- `notifyAll()`

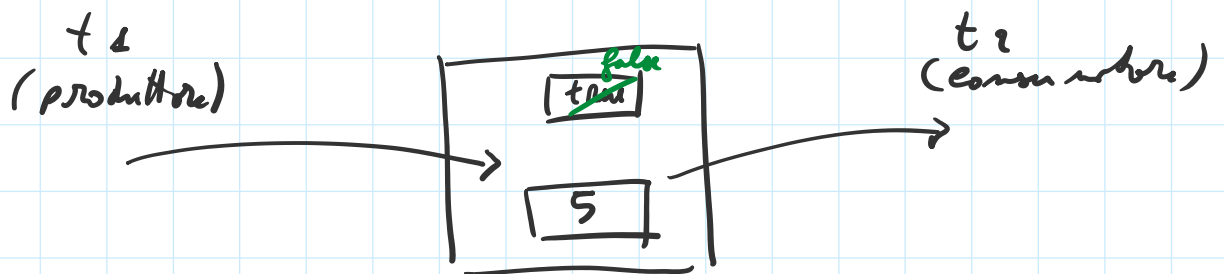
ereditati dalla classe `Object` e quindi disponibili in tutti gli oggetti

Esempio:



Come fa il produttore a capire se il valore precedente è già stato consumato?

Come fa il consumatore a capire se nel buffer c'è un nuovo valore?



I metodi `wait` e `notify` (e `notifyAll`) consentono di sospendere la propria esecuzione e di inviare una notifica

public final void wait() throws InterruptedException
il thread che invoca wait

- sospende la propria esecuzione
- rilascia il lock sull'oggetto
- riacquisisce automaticamente il lock quando esce dalla wait

public final void notify()

risveglia un thread bloccato su una wait

public final void notifyAll()

risveglia tutti i thread bloccati su una wait

Per poter chiamare i metodi wait, notify e notifyAll su un oggetto obbiamo possedere il lock di tale oggetto

Se non possediamo il lock generano

IllegalMonitorStateException

Ad ogni oggetto è associato un "wait-set"

Quando un thread chiama wait su un oggetto il thread viene sospeso, rilascia il lock, viene inserito nel wait-set di tale oggetto

il lock, viene inserito nel wait-set di tale oggetto

Quando eseguo notify su un oggetto un thread a caso tra quelli nel wait-set riceve la notifica

- riacquisisce il lock automaticamente (eventualmente in competizione con altri thread che cercano di prendere lo stesso lock)
- viene riportato in coda pronti

Nel caso della notifyAll la notifica viene inviata a tutti i thread nel wait-set dell'oggetto

In effetti di wait ne esistono più versioni
public void wait(long timeout) throws Interr.

il thread si blocca fino a che

- notify invocata su questo oggetto e il thread è selezionato (a caso)
- notifyAll invocata su questo oggetto
- scatta il timeout (millisecondi)

- invocato il metodo interrupt sul thread

altra versione

```
void wait(long timeout, int nano)
```

```
void wait()
```

↳ corrisponde a

```
wait(0)
```

↑
+ timeout infinito

Schema tipico:

```
class X {
```

```
    synchronized void faiQuandoCompletazione()  
        throws InterruptedException {
```

```
        while (!completazione)
```

```
            wait();
```

```
        // quello che devo fare
```

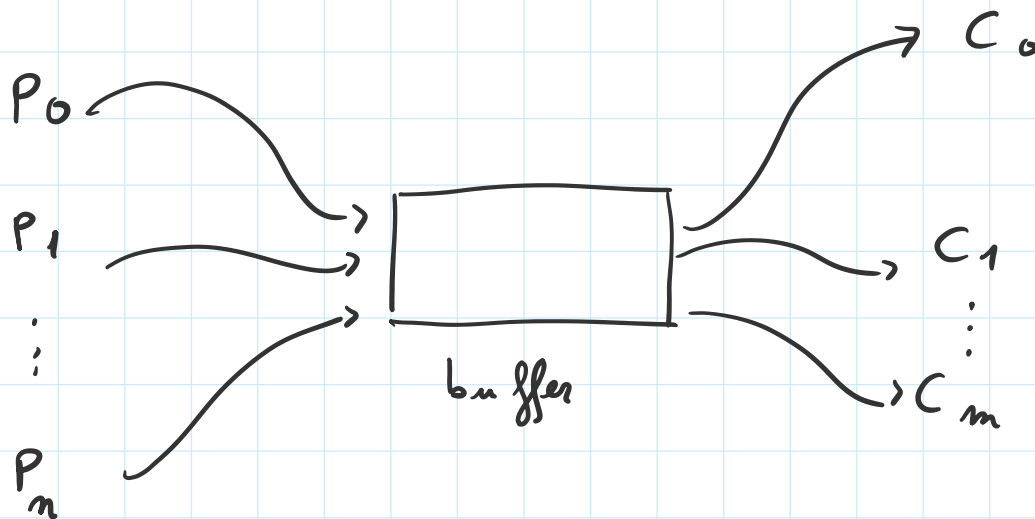
```
    }
```

```

synchronized void cambiaCondizione() {
    // cambia condizione
    notify(); // o notifyAll a seconda
              // dei casi
}
}

```

Produttori consumatori



Ogni produttore ciclicamente

- produce un nuovo valore
- lo immette nel buffer

Se il buffer è pieno deve attendere

Ogni consumatore

- tira fuori un valore dal buffer
- lo consuma

Se il buffer è vuoto deve attendere

buffer

- è in grado di contenere un numero limitato di valori
- è condiviso tra i produttori e i consumatori

```
public class Buffer {
```

```
    private static final int DEFAULT_SIZE = 4;
```

n° di valori che possono essere contenuti in buffer

i valori trasferiti sono stringhe

```
public class Buffer {
```

```
    private static final int DEFAULT_SIZE = 4;
    private String[] vett;
    private int testa, coda;
    private int quanti;
```

```
    public Buffer(){
        this(DEFAULT_SIZE);
    }
```

```
    public Buffer(int n) {
        vett = new String[n];
    }
```

```
    public synchronized void inserisci(String s) throws InterruptedException {
        while(pieno()) {
            System.out.println(Thread.currentThread().getName() + ": mi blocco, buffer pieno");
            wait();
        }
        vett[coda] = s;
        coda = (coda + 1) % vett.length;
        quanti++;
        notifyAll();
    }
```

```
    public synchronized String estrai() throws InterruptedException {
        while(vuoto()) {
            System.out.println(Thread.currentThread().getName() + ": mi blocco, buffer vuoto");
            wait();
        }
        String t = vett[testa];
        testa = (testa + 1) % vett.length;
        quanti--;
        notifyAll();
        return t;
    }
```

```
    public synchronized boolean pieno() {
        return quanti == vett.length;
    }
```

```
    public synchronized boolean vuoto() {
        return quanti == 0;
    }
}
```

```
public class Produttore extends Thread {
```

```
    private Buffer b;
```

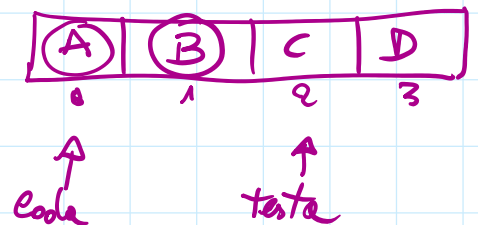
```
    public Produttore(String n, Buffer b) {
        super(n);
```

i valori trasferiti sono stringhe

invoca per le operazioni di estrazione e inserimento

livello di riempimento

resto div. intera



... + ... 1 Thread

```

public Produttore(String n, Buffer b) {
    super(n);
    this.b = b;
}

```

costruttore di Thread

```

public void run(){
    int c = 0;
    try {
        while(true) {
            sleep((long)(Math.random()*1000));
            b.inserisci(getName() + ": " + c++);
            System.out.println(getName() + ": ho inserito");
        }
    } catch (InterruptedException ie) {
        System.out.println("Sono stato interrotto");
        return;
    }
}

```

valore a caso tra 0 e 1 (scelto)
(double)

dorme per un
certo tempo (< 1 s)

```

public class Consumatore extends Thread {

```

```

    private Buffer b;

```

```

    public Consumatore(String n, Buffer b) {
        super(n);
        this.b = b;
    }

```

```

    public void run(){
        try {
            while(true) {
                sleep((long)(Math.random()*1000));
                String s = b.estrail();
                System.out.println(getName() + ": ho estratto \"" + s + "\"");
            }
        } catch (InterruptedException ie) {
            System.out.println("Sono stato interrotto");
            return;
        }
    }
}

```

```

import java.util.Scanner;

```

```

public class Prova {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Quanti produttori? ");
        int p = sc.nextInt();
        System.out.println("Quanti consumatori? ");
    }
}

```

```
int c = sc.nextInt();
System.out.println("Dimensione del buffer? ");
int b = sc.nextInt();

Buffer buf = new Buffer(b);
for(int i=0; i<p; i++)
    new Produttore("Produttore " + i, buf).start();
for(int i=0; i<c; i++)
    new Consumatore("Consumatore " + i, buf).start();
}
}
```